

SIT315 M2.T1P — Parallel Matrix Multiplication

Student documentation: parallelisation / decomposition, then performance evaluation (sequential vs `std::thread` vs OpenMP).

Machine: 13th Gen Intel Core i7-13620H (10 cores / 16 logical processors), 16 GB RAM, Windows 10 Home 64-bit

Compiler: MinGW-w64 g++ 13.2.0, `-O2 -std=c++17` (plus `-pthread` / `-fopenmp`)

Metric: wall-clock time of the multiply kernel only (not initialisation, not file I/O)

Speedup: $S = T_{\text{sequential}} / T_{\text{parallel}}$

Repeats: mean of 3 runs for $N = 800$ and 1600 ; mean of 2 runs for $N = 2400$

1. Problem

Compute $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ where $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ are $N \times N$. $\mathbf{A}$ and $\mathbf{B}$ are filled with random values. For each output element $C[i,j]$, take the dot product of row i of $\mathbf{A}$ and column j of $\mathbf{B}$ (three nested loops). Timing starts after $\mathbf{A}$ and $\mathbf{B}$ are filled and stops before writing $\mathbf{C}$ to a file.

2. Decomposition (partitioning)

Each $C[i,j]$ is independent, so the work is embarrassingly parallel. The chosen split is **row-block data partitioning**: the rows of $\mathbf{C}$ are cut into contiguous blocks, one block per thread.

Thread	Rows of C computed
0	$[0, R)$
1	$[R, 2R)$
...	...
$T-1$	remaining rows (absorbs $N \% T$)

$R = N / T$. Each thread reads $\mathbf{A}$ and $\mathbf{B}$ (read-only) and writes only its own rows of $\mathbf{C}$. Write ranges do not overlap, so **no mutexes or atomics are required**.

Alternatives not used: column blocks (worse write locality in row-major layout); cyclic rows (worse cache locality); 2D tiling (more complex than required); parallelising the k -loop (needs a reduction per element, high overhead).

Sequential vs parallel sub-tasks

Must stay sequential: parse N and thread count; seed RNG (`srand(42)`) so all three programs generate the same A, B); allocate; fill A and B; join / implicit barrier; checksum, print time, write file, free memory.

Can run in parallel: thread t computes its row-block of C with the same three nested loops as the sequential program.

```
[allocate] → [fill A, B] → threads compute disjoint rows of C → [join] → [checksum / write / free]
```

3. Implementations

- `matmul_seq.cpp` — sequential three nested loops.
- `matmul_thread.cpp` — C++ `std::thread`: spawn T workers, each calls `multiplyRows(rowStart, rowEnd)`, then join. Chosen instead of `pthread` because argument passing is type-safe and there is less boilerplate; explicit `join()` makes the barrier obvious.
- `matmul_omp.cpp` — OpenMP: `#pragma omp parallel for schedule(static)` on the outer i-loop. Static schedule reproduces the same contiguous row blocks, so the comparison with `std::thread` is fair. `omp_set_num_threads` sets T; the implicit barrier at the end of the parallel for is the join.

OpenMP is less code. `std::thread` makes the partition visible. Both are submitted.

4. Compilation

```
set PATH=C:\msys64\mingw64\bin;%PATH%
g++ -O2 -std=c++17 matmul_seq.cpp -o matmul_seq.exe
g++ -O2 -std=c++17 -pthread matmul_thread.cpp -o matmul_thread.exe
g++ -O2 -std=c++17 -fopenmp matmul_omp.cpp -o matmul_omp.exe
g++ -O2 -std=c++17 validate.cpp -o validate.exe
```

5. Correctness

All three programs use RNG seed 42. Method: write C plus a checksum (sum of all elements), then compare files with `validate.exe`.

```
validate.exe seq_256.txt thread_256.txt → Overall: PASS
validate.exe seq_256.txt omp_256.txt    → Overall: PASS
```

At N = 256: checksum difference 0, element mismatches 0 / 65536. Timed runs used the same checksum for each N:

N	Checksum
800	1253730689227

1600	10023096198117
2400	33827346238363

Parallel output matches sequential output (bit-exact for this kernel).

6. Input sizes vs this machine

Naive multiply is $O(N^3)$. Three double matrices use $3 \times N^2 \times 8$ bytes. $N = 100 / 1000 / 10000$ was not used for the main table: 100 is too small; 10000 is a stress size.

N	Memory (A+B+C)	Sequential mean	Why this size
800	~15 MB (fits in 24 MB L3)	0.36 s	small but not “a few milliseconds”
1600	~61 MB (past L3)	4.37 s	clear step up in work
2400	~138 MB (DRAM)	145.4 s	cache/memory effects dominate

7. Results

N = 800 (mean of 3)

Implementation	Threads	Mean time (ms)	Speedup
Sequential	1	358.4	1.00×
std::thread	2	175.4	2.04×
OpenMP	2	176.4	2.03×
std::thread	8	67.1	5.34×
OpenMP	8	67.3	5.33×

std::thread	16	53.0	6.76× (best at this N)
OpenMP	16	53.1	6.76×

N = 1600 (mean of 3)

Implementation	Threads	Mean time (ms)	Speedup
Sequential	1	4367.1	1.00×
std::thread	2	2575.9	1.70×
OpenMP	2	2619.9	1.67×
std::thread	8	931.1	4.69× (best at this N)
OpenMP	8	946.4	4.61×
std::thread	16	1261.5	3.46×
OpenMP	16	1230.8	3.55×

Weak / negative scaling: at N = 1600, going from 8 → 16 threads slows down (4.69× down to 3.46× for std::thread). Extra threads hurt.

N = 2400 (mean of 2)

Implementation	Threads	Mean time (ms)	Speedup
Sequential	1	145380.5 (~145 s)	1.00×
std::thread	2	60802.0	2.39×
OpenMP	2	61046.2	2.38×

std::thread	8	19330.6	7.52×
OpenMP	8	18409.0	7.90×
std::thread	16	11339.2	12.82× (best overall)
OpenMP	16	12341.0	11.78×

8. OpenMP vs std::thread vs sequential

- Both parallel versions beat sequential at every measured (N, T) pair.
- OpenMP and std::thread stay within a few percent in most cells — same row-block decomposition.
- N = 2400, T = 8: OpenMP slightly faster (7.90× vs 7.52×).
- N = 2400, T = 16: std::thread slightly faster (12.82× vs 11.78×). OpenMP's second 16-thread run was slower (13.6 s vs 11.1 s); the mean includes that outlier.

Best configuration on this PC: **N = 2400, 16 threads, std::thread → 12.82×**.

9. Why performance changes

Thread overhead and scheduling. At N = 800, 2 threads give almost ideal 2.04×. 16 threads still help (6.76×) but efficiency is only ~42% because fork/join is a larger fraction of a 0.36 s run.

At N = 1600, T = 16, speedup drops vs T = 8. This CPU is 6 P-cores + 4 E-cores (16 logical processors). Static equal chunks give E-cores and SMT siblings the same row count as P-cores; the barrier waits for the slowest worker. Sixteen writers also increase contention on B (naive i-j-k streams down columns of B).

At N = 2400, 16 threads win (12.82×, ~80% efficiency) because the sequential kernel is memory-starved and extra occupancy helps.

Memory / cache cliff. If time scaled only with N^3 , N = 2400 should be about 3.4× N = 1600. Sequential time instead went 4.37 s → 145 s (~33×). N = 800 fits in L3 (~15 MB); N = 1600 and 2400 do not. Parallel row blocks do not fix the B access pattern, but several cores can issue memory requests at once.

Run-to-run variation. Results are means. N = 800, std::thread T = 8 ranged 58.6–73.6 ms. An idle sequential N = 2400 probe was 82 s; the two sweep runs after load were 143 s and 148 s (turbo/thermals). The table uses the repeated sweep means.

10. Stress test (DNF)

Command: `matmul_seq.exe 30000`

Waited: about 2 minutes 20 seconds, then interrupted.

Observed: no Time: line. Working set already ~7.2 GB and still growing.

Reason: three 30000×30000 double matrices need ~21.6 GB, more than 16 GB RAM. DNF — user interrupted due to long wait / memory pressure.

11. Conclusions

1. Row-block decomposition is correct (checksum and element-wise compare PASS) and lock-free.
2. More threads is not always faster: at $N = 1600$, 16 threads are worse than 8.
3. Larger N makes parallelism more worthwhile here, up to 12.8× at $N = 2400$ with 16 threads, but sequential time collapses when the working set leaves cache.
4. OpenMP and `std::thread` perform similarly; OpenMP wins on code size, `std::thread` on making the partition explicit.
5. Input size must match the machine: too small hides speedup, too large DNFs on RAM or time.

Paste compile / run / validate / DNF screenshots after this page before uploading if OnTrack expects evidence in the PDF.